

KCPC-CS150

Problem Sheet

Dynamic Programming and DP on Trees

Developed by the KCPC Internal Committee, this problem sheet accompanies the *Dynamic Programming* workshop. Problems are divided into six categories:

- Warm-up problems (basic DP state design and tree DP intuition)
 - Core workshop practice set (standard DP transitions, tree DP, and rerooting)
 - Additional practice problems (classic one-dimensional, interval, and grid DP)
 - DP contest collections (systematic revision)
 - Extended homework (advanced state design and optimisation)
 - Extra notes and reading material
-

I. Warm-up Problems

These problems introduce the basic idea of dynamic programming and simple tree DP.

1. **AtCoder Educational DP Contest — P. Independent Set**
(Tree DP basics)

https://atcoder.jp/contests/dp/tasks/dp_p

2. **CSES — Tree Diameter**
(DFS on trees and path-based DP intuition)

<https://cses.fi/problemset/task/1131/>

3. **AtCoder Educational DP Contest — O. Matching**
(Bitmask DP warm-up)

https://atcoder.jp/contests/dp/tasks/dp_o

II. Core Workshop Practice Set

These problems should be attempted during the workshop.

They focus on standard DP transitions, tree DP, and rerooting techniques.

1. **CSES — Tree Distances I**
(Tree DP / rerooting starter)

<https://cses.fi/problemset/task/1132/>

2. **CSES — Tree Distances II**
(Sum of distances on trees)

<https://cses.fi/problemset/task/1133/>

3. **CSES — Subtree Queries**
(Euler tour + subtree aggregation)
<https://cses.fi/problemset/task/1137/>
 4. **LeetCode — Binary Tree Maximum Path Sum**
(Tree DP for best path value)
<https://leetcode.com/problems/binary-tree-maximum-path-sum/>
 5. **LeetCode — House Robber III**
(Tree DP with two states)
<https://leetcode.com/problems/house-robber-iii/>
 6. **LeetCode — Count Good Nodes in Binary Tree**
(DFS state propagation)
<https://leetcode.com/problems/count-good-nodes-in-binary-tree/>
 7. **LeetCode — Longest Univalue Path**
(Path DP on trees)
<https://leetcode.com/problems/longest-univalue-path/>
 8. **LeetCode — Diameter of Binary Tree**
(Classic tree DP)
<https://leetcode.com/problems/diameter-of-binary-tree/>
 9. **LeetCode — Sum of Distances in Tree**
(Rerooting DP)
<https://leetcode.com/problems/sum-of-distances-in-tree/>
 10. **LeetCode — Minimum Height Trees**
(Tree centering / breadth-first reduction)
<https://leetcode.com/problems/minimum-height-trees/>
-

III. Additional Practice Problems

These are good for strengthening your DP modelling and transition design.

1. **LeetCode 1137 — N-th Tribonacci Number**
(1D DP)
<https://leetcode.com/problems/n-th-tribonacci-number/description/>
2. **LeetCode 118 — Pascal's Triangle**
(Combinatorial DP)
<https://leetcode.com/problems/pascals-triangle/description/>
3. **LeetCode 1025 — Divisor Game**
(Game DP)
<https://leetcode.com/problems/divisor-game/description/>
4. **Codeforces 699C — Vacations**
(State DP)
<https://codeforces.com/problemset/problem/699/C>

5. **Codeforces 706C — Hard problem**
(String DP / transition design)
<https://codeforces.com/problemset/problem/706/C>
 6. **Codeforces 607B — Zuma**
(Interval DP)
<https://codeforces.com/problemset/problem/607/B>
 7. **LeetCode 221 — Maximal Square**
(Grid DP)
<https://leetcode.com/problems/maximal-square/description/>
 8. **LeetCode 1039 — Minimum Score Triangulation of Polygon**
(Interval DP)
<https://leetcode.com/problems/minimum-score-triangulation-of-polygon/description/>
-

IV. DP Contests

Recommended for systematic practice.

- **AtCoder Educational DP Contest**
<https://atcoder.jp/contests/dp/tasks>
 - **CSES — Dynamic Programming**
<https://cses.fi/problemset/list/>
-

V. Extended Homework

Important. The following problems are significantly more difficult. They should only be attempted after completing all problems in the warm-up and core sections.

These exercises introduce more advanced tree DP, state optimisation, and problem modelling.

Kutay's Note. If you have any questions about the problems or underlying concepts, feel free to contact me or the Director of Competitive Programmes (Marcell).

1. **Codeforces 161D — Distance in Tree**
(Tree DP + counting pairs by distance)
<https://codeforces.com/problemset/problem/161/D>
2. **Codeforces 461B — Appleman and Tree**
(Tree DP with colour states)
<https://codeforces.com/problemset/problem/461/B>
3. **Project Euler 186 and 265**
(Advanced combinatorics and DP-style reasoning)
<https://projecteuler.net/problem=186>
<https://projecteuler.net/problem=265>

VI. Extra Notes and Reading Material

Extra Notes.

- Always define the state before you start coding.
- Identify the transition, base cases, and the order in which states are computed.
- For tree DP, root the tree and process children before parents.
- For rerooting, compute one downward pass and one upward pass.
- If the state space is large, look for symmetry, pruning, or a smaller representation.
- Memoization is often easiest for top-down DP; tabulation is often easier to debug.
- When a problem says *tree*, think about whether the answer is rooted, unrooted, or needs rerooting.

Reading Material.

- Codeforces blog on DP techniques: <https://codeforces.com/blog/entry/20935>
- Review the official editorials for the AtCoder Educational DP Contest after solving each task.
- Revisit the CSES tree problems with a focus on deriving the state and transition yourself.